



Volume 3 - Commandes du simulateur

Génération, transport, messages, bruit, signaux et captures STGF

Ce volume suit `stgs_satellite_simulator` depuis le parsing jusqu'à la socket et détaille la différence entre perte, corruption CRC et bruit de mesure.

Table des matières

1	Chemin commun	2
2	Trafic aléatoire TCP	2
2.1	Création d'une trame aléatoire	2
3	Trafic aléatoire UDP	2
4	Découverte d'un listener TCP	2
5	Envoi d'un message texte	3
6	Envoi d'une onde sinusoïdale bruitée	3
6.1	Génération	3
6.2	Pourquoi le bruit est ajouté avant CRC	4
6.3	Réception et filtre	4
7	Perte simulée vs corruption de transport	4
7.1	-loss 0.05	4
7.2	-corrupt 0.02	4
8	Créer uniquement un fichier de replay	4
9	RNG et reproductibilité	5
10	Table complète des limites simulateur	5

1 Chemin commun

Point d'entrée du simulateur

```
main → parseArgs → run(Options) → TerminalUi → discoverPort éventuel → RNG  
mt19937 → FrameFileWriter éventuel → socket TCP/UDP éventuelle → boucle count  
→ makeFrame → encodeFrame → loss/corrupt → send / capture → sleep_until →  
résumé
```

Source : apps/satellite_simulator.cpp, lignes 14–26; **Source :** apps/satellite_simulator/SimulatorApp.cpp, lignes 103–200

2 Trafic aléatoire TCP

```
./build/stgs_satellite_simulator --tcp --host 127.0.0.1 --port 9000 --count 1000 --rate  
500
```

`parseArgs()` valide TCP, IPv4 à la construction de l'adresse, port, nombre de trames et rate fini positif ou nul. `run()` crée un `std::mt19937`, ouvre une socket `SOCK_STREAM`, construit l'adresse et appelle `connect()` une fois. Ensuite chaque trame est envoyée avec `sendAll()`, qui boucle jusqu'à ce que tous les octets aient été transmis; `MSG_NOSIGNAL` évite qu'un peer fermé tue le processus par `SIGPIPE`.

Source : apps/satellite_simulator/SimulatorSocket.cpp, lignes 61–108

2.1 Création d'une trame aléatoire

`makeFrame()` applique les valeurs de démonstration suivantes : température gaussienne 22°C, $\sigma = 4$; batterie 100..0 cyclique; statuts pondérés 90% NOMINAL, 7% WARNING, 2% CRITICAL, 1% SAFE_MODE; payload binaire uniforme de la taille demandée.

Nature des distributions

Ces valeurs servent à exercer le code et la logique de santé. Elles ne représentent aucune mission ou spécification Safran.

3 Trafic aléatoire UDP

```
./build/stgs_satellite_simulator --udp --host 127.0.0.1 --port 9000 --count 1000 --rate  
500
```

Le chemin de génération est identique. La socket est `SOCK_DGRAM` et chaque trame complète est envoyée par un unique `sendto()`. Le code exige que le nombre d'octets retourné soit exactement la taille de la trame.

Source : apps/satellite_simulator/SimulatorApp.cpp, lignes 83–99, 161–170

4 Découverte d'un listener TCP

```
./build/stgs_satellite_simulator --tcp --host 127.0.0.1 --discover-ports 9000:9010 --count  
100
```

`-discover-ports` est limité au loopback IPv4 et à 256 ports. `discoverPort()` appelle `scanTcpPorts()`, affiche le rapport et choisit le premier état OPEN.

Découverte

```
scanTcpPorts → probeTcpPort pour chaque port → socket non bloquante
→ connect → si EINPROGRESS : poll(POLLOUT) → getsockopt(SO_ERROR) →
OPEN/CLOSED/TIMEOUT/ERROR → premier OPEN choisi → vraie socket du simulateur
+ connect
```

Limite de la découverte

OPEN signifie seulement qu'un listener TCP a accepté la connexion. STGS v1 est unidirectionnel et n'a pas de handshake d'identité; la découverte ne prouve donc pas que le service est réellement STGS.

5 Envoi d'un message texte

```
./build/stgs_satellite_simulator --tcp --host 127.0.0.1 --port 9000 --message "Bonjour
STGS"
```

Sans `-count`, le mode message choisit une seule trame.

Encapsulation du message

```
makeFrame(sequence=0) → encodeTextMessagePayload → STGA magic + version + kind
TEXT_MESSAGE + sequence + texte → frame.payload → encodeFrame → header STGS +
payload STGA + CRC32 → sendAll/sendto
```

À la réception : `decodeFrame()` valide l'enveloppe externe, puis `decodeApplicationPayload()` retrouve `TEXT_MESSAGE`. `TerminalUi::receivedMessage()` affiche le texte après `sanitizeRemoteText()` : les caractères de contrôle, notamment ESC, sont neutralisés.

Taille du message

Le payload STGS est limité à 4096 octets. Le header STGA message occupe 10 octets; le texte peut donc occuper au maximum 4086 octets. La chaîne est transportée en octets; le code ne valide pas formellement l'UTF-8.

6 Envoi d'une onde sinusoïdale bruitée

```
./build/stgs_satellite_simulator --tcp --host 127.0.0.1 --port 9000 \
--signal --signal-frequency 5 --sample-rate 200 \
--signal-samples 256 --signal-amplitude 1.0 --noise-stddev 0.35 \
--count 5 --rate 2
```

6.1 Génération

Pour le bloc b , `makeSignalPayload()` calcule :

$$n_0 = b \times N, \quad \omega = 2\pi f / f_s,$$

$$x[n] = A \sin(\omega(n_0 + n)) + \varepsilon[n], \quad \varepsilon \sim \mathcal{N}(0, \sigma^2).$$

L'index absolu `startSampleIndex` maintient la continuité de phase entre blocs.

Source : `apps/satellite_simulator/SimulatorFrameFactory.cpp`, lignes 63–97

6.2 Pourquoi le bruit est ajouté avant CRC

`-noise-stddev` modifie les **données de mesure**, puis le payload et le CRC sont calculés sur ces valeurs. La trame est donc intègre mais son signal est bruité : c'est un cas destiné au DSP.

6.3 Réception et filtre

Après décodage STGA, la station choisit :

- `none` : aucune transformation ;
- `moving-average` : FIR centré ;
- `sine-projection` : projection sur sin/cos à la fréquence nominale + composante DC.

Elle calcule ensuite RMS, résidu, amplitude estimée et $20 \log_{10}(RMS_{filtré}/RMS_{résidu})$.

Limites signal STGA v1

- sample rate : 1.65535 Hz (u16) ;
- fréquence : finie, strictement $0 < f < f_s/2$ (Nyquist) ;
- amplitude : finie et ≥ 0 ;
- bruit sigma : fini et ≥ 0 ;
- header signal STGA : 22 octets ;
- chaque sample : float32 = 4 octets ;
- samples par trame : $\lfloor (4096 - 22)/4 \rfloor = 1018$ maximum ;
- `startSampleIndex` : u32, donc le run complet doit rester dans sa plage.

7 Perte simulée vs corruption de transport

```
./build/stgs_satellite_simulator --tcp --host 127.0.0.1 --port 9000 --loss 0.05 --corrupt 0.02 --count 1000
```

7.1 `-loss 0.05`

Après `encodeFrame()`, le RNG décide de laisser tomber la trame. Elle n'est ni envoyée ni écrite dans la capture STGF et incrémente `dropped`.

7.2 `-corrupt 0.02`

Si la trame n'est pas perdue, `maybeCorrupt()` choisit un octet aléatoire **après calcul du CRC** et applique XOR 0x5A. La trame est ensuite envoyée/capturée. Le but est de produire un CRC incohérent et de vérifier le chemin de rejet côté station.

Pourquoi 0x5A ?

Le masque binaire 01011010 retourne plusieurs bits sans forcer l'octet vers une valeur fixe. C'est un choix de simulation nommé, pas une constante du protocole.

8 Créer uniquement un fichier de replay

```
./build/stgs_satellite_simulator --output-file frames.stgf --count 1000 --seed 42
```

Aucun transport n'est requis si `-output-file` est présent. `run()` crée un `FrameFileWriter`, génère les trames, applique perte/corruption comme d'habitude, puis écrit les trames effectivement produites au format STGF.

Capture STGF

`FrameFileWriter` ctor → écrit magic STGF + version → `writeFrame` → longueur u32 + octets STGS complets → flush vérifié

9 RNG et reproductibilité

`-seed N` fixe `std::mt19937`. À configuration et environnement identiques, payloads aléatoires, bruit, pertes, corruptions et statuts suivent la même séquence pseudo-aléatoire. L'horodatage `timestampMs`, lui, provient de l'horloge système et reste variable.

10 Table complète des limites simulateur

Option	Borne	Remarque
<code>-count</code>	1..10 000 000	Garde-fou CLI.
<code>-rate</code>	≥ 0 , fini	0 = pas de temporisation.
<code>-satellite</code>	0..65535	u16 du protocole.
<code>-payload-size</code>	0..4096	Mode aléatoire seulement.
<code>-loss</code> , <code>-corrupt</code>	[0,1]	Probabilités.
<code>-discover-ports</code>	≤ 256 ports	Loopback IPv4 seulement.
Timeout découverte	1..60000 ms	Par port.
Message	≤ 4086 octets	Dérivé de <code>MaxPayloadSize</code> .
Signal samples	1..1018	Dérivé du header STGA.